



软硬件对接说明文档

文档类型	技术规格
状态	草稿

智能书包 SmartBag 软硬件对接说明

文档类型：技术对接说明

状态：可联调版

受众：嵌入式固件工程师（ESP32 / ESP32-S3 / ESP32-CAM）

适用范围：当前 SmartBag 网页 + MQTT Broker + 设备联调

0. 核心目标：让设备稳定接入系统

这份文档的核心任务只有一个：帮助硬件设备稳定、顺畅地接入当前的 SmartBag 系统。

为了方便理解，我们可以把整个系统拆分为 3 条主要链路：

1. 网页前端链路

- 网址：`https://bag.versecraft.cn/location`
- 说明：这是提供给最终用户访问的网页，不是给硬件直接连接的 MQTT 地址。

2. 硬件 MQTT 链路（ESP32 → Broker）

- 连接地址：`mqtt.bag.versecraft.cn`
- 端口：`1883`
- 说明：硬件设备请务必走 TCP MQTT 协议。

3. 网页 MQTT 链路（浏览器 → Broker）

- 连接地址：`wss://mqtt.bag.versecraft.cn`
- 说明：浏览器走的是 WSS 协议。这条链路是网页端内部使用的，千万不要让硬件去连这个地址。

💡 避坑指南：

- 硬件**绝对不要**连接 `wss://mqtt.bag.versecraft.cn` !
- 硬件**只能**连接 `mqtt.bag.versecraft.cn:1883` !
- 记住：浏览器用 WSS，硬件用 TCP，井水不犯河水。

🔧 1. 基础网络配置（硬件必看）

1.1 MQTT 连接参数一览

配置项	设定值	说明
Broker Host	<code>mqtt.bag.versecraft.cn</code>	⚠ 不要带 <code>http://</code> 或 <code>https://</code>
Broker Port	<code>1883</code>	原生 MQTT TCP 端口
Protocol	MQTT v3.1.1 或 MQTT v5	二选一即可
Client ID	<code>bag_esp32_<MAC后6位></code>	必须全局唯一，否则会互相踢下线
Username	留空	当前默认匿名接入
Password	留空	当前默认匿名接入
KeepAlive	<code>60s</code>	推荐心跳间隔

1.2 再次强调：硬件与网页的区别

- 对于硬件（你的主战场）：
 - 请使用 `mqtt.bag.versecraft.cn:1883`
 - 协议是 **TCP MQTT**
 - ❌ 不要用 HTTPS
 - ❌ 不要用 WebSocket
 - ❌ 不要在 Host 前面加 `http://`
- 对于网页：
 - 网页内部会使用 `wss://mqtt.bag.versecraft.cn`（这是浏览器的专有连接方式，与硬件无关）。

📡 2. Topic 规范（当前联调主线）

目前的联调工作主要围绕以下几个固定的 Topic 展开：

数据类型	Topic	方向	QoS	Retain	说明
设备状态 / 遗嘱	<code>v5/bag/status</code>	上行 ↑	1	True	告诉云端设备是在线还是离线
传感器数据	<code>v5/bag/sensors</code>	上行 ↑	0	False	比如温湿度、电量（高频，偶尔丢包没关系）

数据类型	Topic	方向	QoS	Retain	说明
GPS 定位	v5/bag/gps	上行 ↑	0	False	实时位置更新
云端控制	v5/bag/cmd	下行 ↓	1	False	云端下发的指令，设备必须收到
指令回执	v5/bag/cmd/ack	上行 ↑	1	False	设备执行完指令后，给云端一个确认

QoS 与 Retain 小贴士：

- **QoS 0**：适合 GPS、传感器等高频发送的数据，偶尔丢一两条不影响大局。
- **QoS 1**：适合状态、控制指令、回执等重要数据，必须保证送达。
- **Retain=True**：建议只给 v5/bag/status 开启。这样当网页刚打开并订阅时，能立刻知道设备当前的在线状态。

3. 上行协议：设备发给云端 (ESP32 -> Cloud)

核心原则：所有上报的 Payload（数据包）必须是标准 JSON 格式。

3.1 设备在线 / 离线状态 (LWT)

Topic: v5/bag/status

- 设备连接成功后，请立即发送：

```
{
  "status": "online"
}
```

- 设备连接前，请设置遗嘱 (Last Will)：

```
{
  "status": "offline"
}
```

(建议 QoS: 1, Retain: True。这样设备断电或异常掉线时，网页能自动显示离线。)

3.2 传感器数据

Topic: v5/bag/sensors

建议频率：每 3~5 秒发送一次即可。

Payload 示例：

```
{
  "battery": 88,
  "temp": 24.5,
  "humid": 45
}
```

(字段说明: `battery` 是电量, 建议 0-100 的整数; `temp` 是温度; `humid` 是湿度。请尽量不要修改字段名, 因为前端目前是按这些名字读取的。)

3.3 GPS 定位数据

Topic: `v5/bag/gps`

建议频率: 移动中 1秒/次, 静止时 5~10秒/次。

Payload 示例:

```
{
  "lat": 31.230412,
  "lng": 121.473701
}
```

千万注意:

- 必须上传 WGS-84 原始坐标!
- 不要在硬件端自己转换成 GCJ-02 坐标!
- 地图偏移的问题交由前端统一处理。如果硬件自己做转换, 极易导致地图上点位错乱、轨迹乱跳。

3.4 指令回执 ACK

Topic: `v5/bag/cmd/ack`

用途: 收到云端下发的指令并执行完毕后, 回传执行结果。

Payload 示例:

```
{
  "cmd_id": "abc-123",
  "status": 0,
  "msg": "OK"
}
```

(说明: `cmd_id` 请填入云端指令里的 ID; `status = 0` 表示成功, 非 0 表示失败; `msg` 是简短的文字说明。)



4. 下行控制协议: 云端发给设备 (Cloud -> ESP32)

Topic: `v5/bag/cmd`

硬件收到指令后，请解析 JSON 中的 `action` 和 `value` 字段。

4.1 模式切换

```
{
  "id": "abc-123",
  "action": "mode_switch",
  "value": "focus_mode"
}
```

(例如：收到 `focus_mode` 时切换到专注模式。未来可能会新增其他模式，但数据结构会保持不变。)

4.2 屏幕文字显示

```
{
  "id": "abc-124",
  "action": "screen_text",
  "value": "该喝水啦！"
}
```

(硬件收到后，请直接将 `value` 里的内容显示在屏幕上。如果文字太长，硬件可以自行做截断或滚动显示。)

5. 视觉链路（视频流建议）

这部分我们不设硬性规定，从最简单的开始做起：

5.1 第一阶段：局域网 MJPEG 流（当前推荐）

目前前端最容易接入的是局域网直连流：

`http://<ESP32_IP>:81/stream`

- **要求：**输出标准 MJPEG 流，保持可访问，浏览器能直接拉取。
- **目标：**先把局域网里的视频流通，再去考虑公网访问。

5.2 进阶：公网访问建议

未来如果需要在外面也能看视频，**不建议**让摄像头自己去搞定公网 IP。

更科学的做法是：

1. 摄像头乖乖在局域网里工作。
2. 用旁路设备（比如树莓派、小主机）做转发。
3. 服务器再统一反向代理成公网地址，例如：`https://stream.bag.versecraft.cn/stream`

5.3 关于“图片快照上传”

在当前的联调阶段，HTTP POST 上传图片不是必做项。

我们现在的首要任务是：跑通 MQTT + GPS + 传感器 + 局域网视频流。

6. AI 与 Coze 接入（当前可延后）

当前建议：先搞定 MQTT 基础数据，AI 功能放到下一阶段。

如果不影响主链路，这部分可以先跳过。

未来如果接入 AI，返回的 JSON 结构大致如下：

```
{
  "reply": "好的，我会提醒主人注意坐姿。",
  "emotion": "blink",
  "cmd": "LIGHT_FOCUS"
}
```

- `reply`：语音播报的文字。
- `emotion`：屏幕要显示的表情（如 `smile`, `blink`, `think`, `normal`）。
- `cmd`：控制外部设备的动作（如 `LIGHT_FOCUS`, `LIGHT_OFF`, `NONE`）。

7. 硬件联调 Check List（按优先级执行）

请按照以下顺序，一步步打怪升级：

1. [] **MQTT 连接测试**：用 MQTTX 测试连接 `mqtt.bag.versecraft.cn:1883` 是否成功。
2. [] **在线/离线测试**：发送 `{"status":"online"}` 看网页是否亮起，断网看是否触发遗嘱 `{"status":"offline"}`。
3. [] **传感器测试**：发送温湿度电量，看网页仪表盘是否跳动。
4. [] **GPS 测试**：发送真实的 WGS-84 坐标，看地图定位是否准确。
5. [] **指令测试**：通过云端下发文字指令，看设备能否执行并回传 ACK。
6. [] **视频流测试**：在局域网浏览器里打开 `http://<ESP32_IP>:81/stream` 确认有画面。

8. 硬件开发避坑指南（血泪教训）

1.  **别填错地址**：硬件 MQTT 地址是 `mqtt.bag.versecraft.cn`，端口 `1883`。别加 `http://`，也别连网页的 `wss://`。
2.  **Client ID 必须唯一**：建议用 `bag_esp32_<MAC后6位>`，否则设备会疯狂互相踢下线。
3.  **别自己转坐标**：老老实实发 WGS-84，别自作聪明转 GCJ-02，不然地图会满天飞。
4.  **用正经 JSON 库**：千万别自己用字符串拼接 JSON，少个逗号少个引号会查到怀疑人生（推荐 ArduinoJson 或 cJSON）。
5.  **一定要写自动重连**：必须包含 **WiFi 断线重连** 和 **MQTT 断线重连**。不然现场一掉线就彻底失联，很难排查。

🎯 10. 给硬件同学的一句话总结

千言万语汇成 4 句话，请牢记：

1. **MQTT 只连：** `mqtt.bag.versecraft.cn:1883`
2. **Topic 就这几个：** `v5/bag/status`、`v5/bag/sensors`、`v5/bag/gps`、`v5/bag/cmd`、`v5/bag/cmd/ack`
3. **GPS 坐标：**只发 WGS-84 原始坐标，别自己乱转。
4. **视频流：**先在局域网里把 `http://<ESP32_IP>:81/stream` 跑通。

先把这 4 件事搞定，我们再聊别的！加油！🚀